

0.79.1. Suivre une autre lettre — de plus en plus vite

Le 0.79, mais le B accélère : toutes les 3 secondes, il attend un peu moins entre deux pas. Facile au début, de plus en plus dur.

1. Ce qu'il faut comprendre

C'est le 0.79, avec un B qui accélère. Au début, il est lent (600 ms par pas) : on lui échappe facilement. Toutes les 3 secondes, il devient un peu plus rapide, jusqu'à 200 ms par pas : là, il est presque impossible à fuir.

Ce qui est nouveau ici : une vitesse qui CHANGE pendant le jeu. `chaque(600)` ne peut pas le faire : son nombre est écrit en clair, traduit avant le jeu, et ne bouge plus. On compte donc les images nous-mêmes : `compte` augmente à chaque image, et quand il atteint `lenteur`, le B fait un pas.

`lenteur` est le nombre d'images entre deux pas : 36 au départ (600 ms). Toutes les 3 secondes, `if` (`chaque(3000)`), on lui retire 6 : 30, 24, 18... jusqu'à 12 (200 ms), le plus rapide. Le `if (lenteur > 12)` empêche de descendre plus bas.

`compte >= lenteur` (plus grand **ou égal**) : si `lenteur` diminue juste quand `compte` était déjà plus grand, le pas se fait quand même, au lieu d'attendre que `compte` fasse le tour des 255.

En bas à droite, `lenteur` s'affiche : on la voit descendre, 036, 030, 024...

Les fonctions de cette leçon — et le tuto de chacune

- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1670 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          92 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur()  124 o ← main() — une fonction fournie par le langage
deplace_croix()    217 o ← main() — une fonction fournie par le langage
un_pas()           117 o ← main() — une fonction fournie par le langage
main()             214 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
```

AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre	– attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), poser()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	704 o	← CopierTuiles	– la police entière
"PRIS"	4 o	← texte()	– un texte gravé

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

effacer()	× 2	30 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o), EffacerCases (12 o)
poser()	× 2	26 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o)
bouton()	× 4	20 o sur place, dans deplace_croix() + LireManette (37 o)
texte()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
nombre()	× 1	10 o sur place, dans main() + EcrireNombre (33 o)
images()	× 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

un_pas()	× 1	23 o sur place, dans main()
deplace_croix()	× 1	19 o sur place, dans main()
chaque_minuteur()	× 1	10 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <chaque>           // répond 1 toutes les n millisecondes
#include <un_pas>            // fait faire un seul pas à une tuile
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <nombre>           // écrit un nombre en chiffres
#include <texte>            // écrit un texte à l'écran
```

2. UNE VARIABLE — X

```
uint8_t x = 10;           // le A (la croix)
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — BX

```
uint8_t bx = 0;           // le B (le poursuivant)
```

5. UNE VARIABLE — BY

```
uint8_t by = 0;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;           // la direction du prochain pas du B
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;
```

8. UNE VARIABLE — LENTEUR

```
uint8_t lenteur = 36; // les images entre deux pas du B : 36 = 600 ms au départ
```

9. UNE VARIABLE — COMPTE

```
uint8_t compte = 0; // les images comptées depuis son dernier pas
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        // Toutes les 3 secondes, le B accélère : 6 images de moins entre deux pas.
        if (chaque(3000)) {
            if (lenteur > 12) { // 12 images (200 ms) : le plus rapide
                lenteur = lenteur - 6;
            }
        }

        // Le pas du B, quand compte atteint lenteur.
        compte++;
        if (compte >= lenteur) {
            compte = 0;
            sx = 0;
            if (x > bx) { sx = 1; }
            if (x < bx) { sx = -1; }
            sy = 0;
            if (y > by) { sy = 1; }
            if (y < by) { sy = -1; }
            if (sx != 0 || sy != 0) {
                un_pas(bx, by, ALPHABET[1], sx, sy);
            }
        }

        deplace_croix(x, y, ALPHABET[0], 250); // le A, par-dessus
        nombre(17, 17, lenteur); // la lenteur du B : elle descend

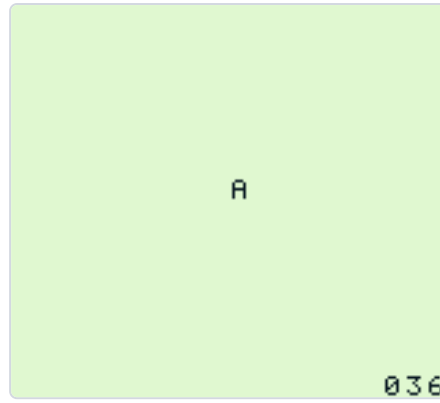
        if (bx == x && by == y) {
            texte(0, 17, "PRIS");
        }
    }
}
```

3. Ce qui se passe, étape par étape

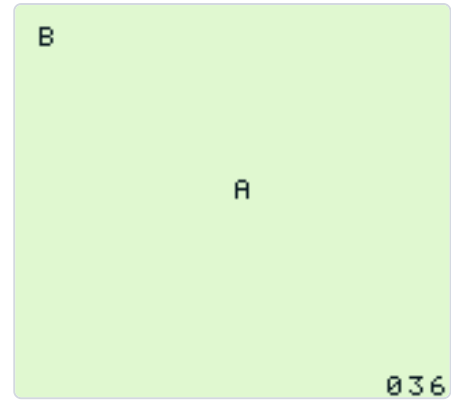
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



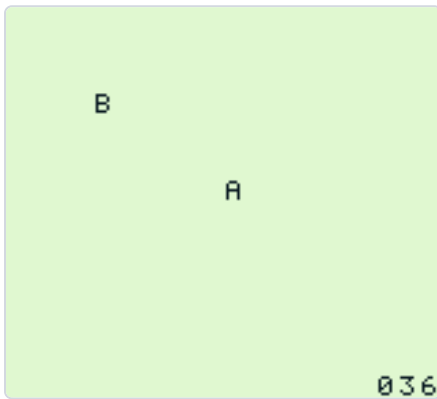
la toute première image — la console vient de démarrer — image 1



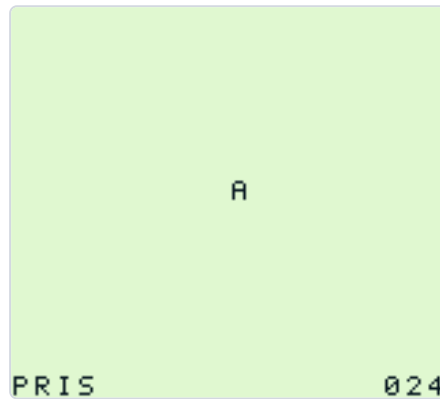
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Comme au 0.79, mais le B accélère toutes les 3 secondes ; en bas à droite, sa lenteur descend de 036 à 012.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, 36 images entre deux pas
- ✓ elle diminue avec le temps (36 → 24)
- ✓ elle s'arrête à 12, le plus rapide
- ✓ immobile, le A finit attrapé

Ensuite : **0.80. x et y rangés ensemble : struct** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++